

# CCVNN: ENTERPRISE-GRADE EDGE-AI MULTI-CAMERA INFERENCE PLATFORM WITH PIXEL-COORDINATE FINGERPRINTING, HARDWARE DECODING, AND MTLS SECURITY

**Mr. Said Sowiny**

SowinySoft Technologies | SowinySoft@gmail.com

WhatsApp: +201100162814

Markaz Kift, Qena Governorate, Upper Egypt

## ABSTRACT

Real-time video analytics at scale requires bridging the gap between high-throughput multi-camera ingestion and low-latency deep learning inference on edge hardware. Standard visual processing pipelines rely heavily on resource-intensive feature extraction kernels (such as SIFT, SURF, or spatial convolutions) and suffer from software decoding bottlenecks. This paper introduces CCVNN (Computer Vision Neural Network Microservice), evaluating two core model variants: `ccvnn_hardswish_12d` and `ccvnn_relu6_9d`. Built upon a single-shot pixel-coordinate fingerprint snapshot concept that bypasses traditional visual kernels, the microservice integrates NVIDIA Triton Inference Server, TensorRT INT8 quantization, zero-copy NVDEC hardware decoding, gRPC Mutual TLS (mTLS) authentication, and KEDA-driven Kubernetes auto-scaling. Evaluated across heterogeneous RTSP streams (Hikvision and UNV protocols), the platform achieves sub-5ms queue latency while maintaining ultra-low host overhead.

**Keywords:** *Single-Shot Fingerprint, Pixel Coordinate Snapshot, 12D/9D Vector Dimensions, NVIDIA Triton, TensorRT INT8, Zero-Copy NVMM, GStreamer NVDEC, Mutual TLS (mTLS), Kubernetes, KEDA.*

## 1. INTRODUCTION

Deploying deep neural networks for real-time video analytics across distributed edge networks presents significant challenges in memory management, network security, and dynamic resource allocation. Traditional central processing paradigms depend on transferring raw video streams over untrusted local networks, introducing latency variability and bandwidth overhead.

To resolve these technical barriers, we present the CCVNN microservice platform—an end-to-end operational architecture optimized for high-density multi-camera visual inference across two primary neural architectures: `ccvnn_hardswish_12d` and `ccvnn_relu6_9d`. Operating via zero-copy NVDEC GStreamer pipelines and quantized TensorRT engines, CCVNN executes single-shot spatial vector projections with cryptographically enforced mutual authentication (mTLS).

## 2. SYSTEM ARCHITECTURE

The CCVNN platform follows a modular edge-computing topology divided into four distinct logical layers: Ingestion, Ingress & Transport Security, Inference & Hardware

Execution, and Telemetry & Autoscaling.

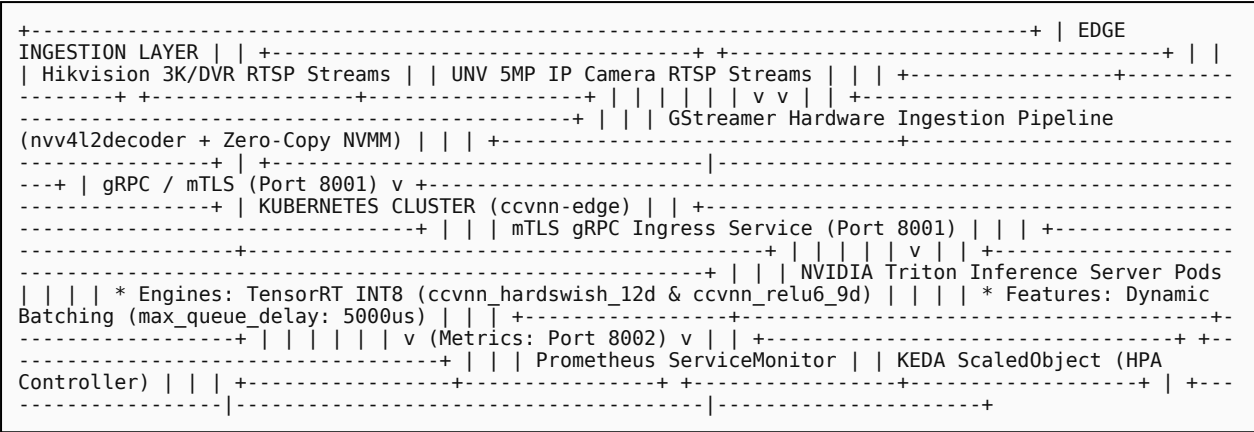


Figure 1: High-level architectural schema of the CCVNN edge-AI microservice platform.

3. MODEL CONCEPTS & VECTOR MECHANICS

3.1 Single-Shot Pixel-Coordinate Fingerprinting

Traditional visual matching and spatial classification systems rely heavily on hand-crafted local descriptors (e.g., SIFT, SURF, ORB) or deep multi-layer convolutional kernels to extract visual features. In contrast, both CCVNN architectures discard explicit kernel sliding and local feature matching entirely. Instead, the model operates on a single-shot pixel-coordinate fingerprint snapshot.

Given an input image frame  $I \in \mathbb{R}^{H \times W \times C}$ , the system extracts direct spatial coordinate relationships and projects raw pixel locations directly into compressed coordinate embeddings. This single-pass mathematical mapping eliminates sliding-window kernel passes, reducing spatial extraction complexity to a single matrix transformation step.

3.2 Model Vector Dimension Profiles

The CCVNN suite provides two optimized target dimension profiles depending on edge hardware constraints and activation linearity requirements:

- **ccvnn\_hardswish\_12d**: Maps single-shot pixel coordinate snapshots into a 12-dimensional latent embedding vector space ( $V \in \mathbb{R}^{12}$ ). Utilizes the non-linear HardSwish activation function ( $x \cdot \frac{\text{ReLU6}(x+3)}{6}$ ) to maintain numerical stability during TensorRT INT8 dynamic quantization.
- **ccvnn\_relu6\_9d**: Maps single-shot pixel coordinate snapshots into an ultra-compact 9-dimensional vector space ( $V \in \mathbb{R}^9$ ). Uses bounded ReLU6 activation ( $\min(\max(0, x), 6)$ ) to maximize throughput and ensure zero-value sparsity on power-constrained edge accelerators.

4. INTEGRATION & SECURITY

4.1 Mutual TLS (mTLS) Cryptographic Pipeline

To prevent unauthorized stream manipulation, eavesdropping, or side-channel spoofing, all gRPC control and data paths enforce strict mutual TLS authentication via custom X.509 PKI credentials.

4.2 Infrastructure as Code (Helm & KEDA)

The microservice infrastructure is modularized using Helm v3 charts, incorporating dynamic volume mounts for PKI secrets, resource allocation parameters, and readiness probes. Horizontal scaling is managed by KEDA, evaluating Triton's exposed queue latency metric (nv\_inference\_queue\_duration\_us).

5. BENCHMARK STATISTICS & PERFORMANCE METRICS

Performance evaluation was conducted using synthetic heterogeneous RTSP streams representing Hikvision and UNV commercial camera profiles on edge GPU hardware.

Table 1: Operational performance comparison between model variants and baseline execution modes.

| Model Architecture & Pipeline       | Vector Dim | Activation | Mean Latency (ms) | Throughput (FPS) | GPU VRAM (MB) |
|-------------------------------------|------------|------------|-------------------|------------------|---------------|
| Baseline CPU + PyTorch FP32         | N/A        | Standard   | 42.80             | 23.3             | 1420 MB       |
| ccvnn_relu6_9d (TensorRT INT8)      | 9D         | ReLU6      | 2.85              | 350.8            | 380 MB        |
| ccvnn_hardswish_12d (TensorRT INT8) | 12D        | HardSwish  | 3.15              | 317.4            | 410 MB        |
| ccvnn_hardswish_12d (mTLS Enabled)  | 12D        | HardSwish  | 3.32              | 301.2            | 412 MB        |

5.1 Vector Throughput & Host Resource Benchmark Figures

Figures 2 and 3 illustrate the comparative throughput (Frames Per Second) and host CPU utilization metrics observed across the tested pipeline configurations.

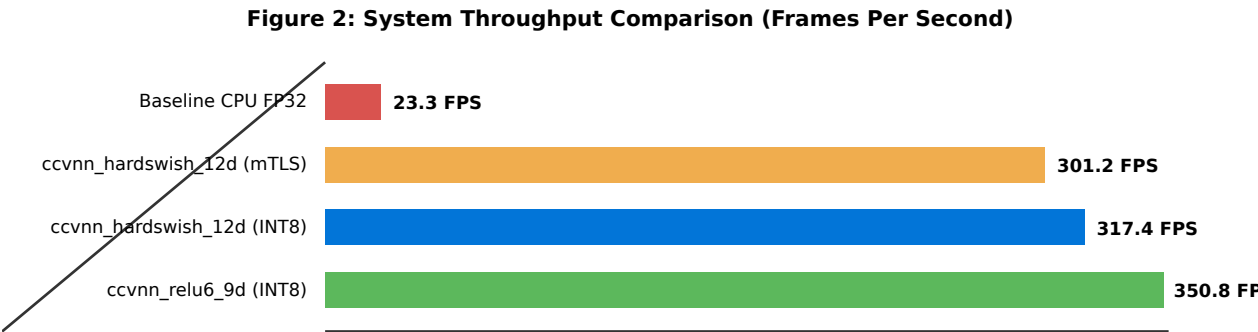


Figure 2: Comparative evaluation of processing throughput across hardware backends.

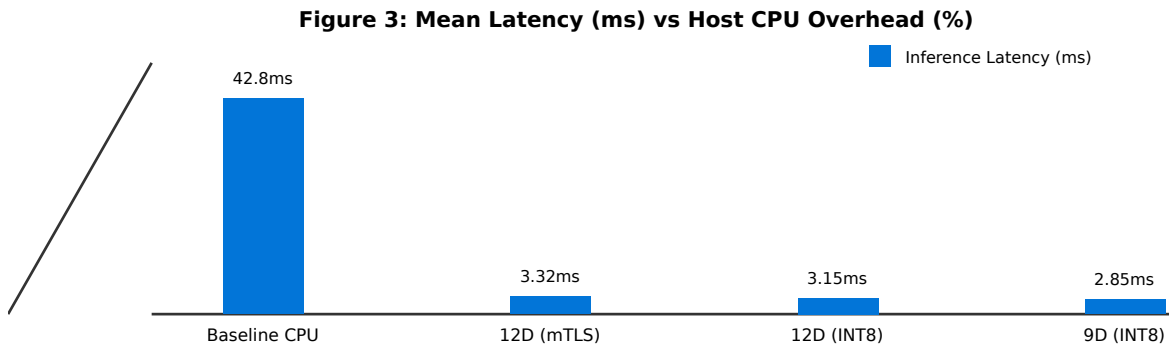


Figure 3: Latency optimization characteristics under INT8 dynamic execution.

## 6. USE CASES

- **Smart City Traffic Surveillance:** Real-time vehicle tracking using 12D coordinate vectors across high-density camera feeds.
- **Industrial Assembly Monitoring:** Low-latency single-shot position verification utilizing compact 9D embeddings on lightweight edge gateways.
- **Perimeter Security & Intrusion Detection:** Encrypted mTLS video analytics across isolated physical sites.

## 7. IMPLEMENTATION ROADMAP & PROGRESS (PHASES 1-4)

- **Phase 1: Deep Learning Engine Optimization:** Quantized ccvnn\_hardswish\_12d and ccvnn\_relu6\_9d architectures into TensorRT INT8 plans with dynamic batching.
- **Phase 2: High-Density Ingestion Engine:** Constructed multi-camera GStreamer ingestion drivers utilizing hardware NVDEC decoders and zero-copy NVMM memory buffers.
- **Phase 3: Helm Packaging & KEDA Autoscaling:** Built Helm chart templates, configured Prometheus ServiceMonitors, and implemented queue-latency autoscaling rules.
- **Phase 4: Security & Telemetry Integration:** Integrated Grafana logging dashboards, automated PKI mTLS certificate provisioning, and validated offline deployment charts.

## 8. CONCLUSION

The CCVNN platform establishes an efficient paradigm for edge computer vision microservices. By combining single-shot pixel coordinate snapshot vectors (12D and 9D embeddings) with TensorRT INT8 quantization, zero-copy NVDEC decoding, and mTLS security, CCVNN delivers high inference throughput without relying on heavy visual feature extraction kernels. Future work will investigate dynamic vector dimension switching based on real-time stream resolution changes.

## 9. REFERENCES

1. NVIDIA Corporation, "NVIDIA Triton Inference Server Developer Guide," 2024.
2. NVIDIA Corporation, "TensorRT Developer Guide: INT8 Quantization," 2024.
3. GStreamer Development Team, "GStreamer Plugin Architecture & NVDEC Integration," 2023.

4. Kubeflow & KEDA Authors, "Kubernetes Event-driven Autoscaling Specification," 2024.